

CKEditor for WordPress — Patch & Upgrade Session

Session date: July 21, 2026

Topic: Patching the "CKEditor for WordPress" plugin for modern WordPress/PHP compatibility, then evaluating and executing a CKEditor core upgrade.

Turn 1 — 19:21

USER

Create a PHP patch for `ckeditor_class.php` that replaces the deprecated `wp_richedit_pre()` call with `format_for_editor()` while maintaining backward compatibility. Implement a conditional check using `function_exists('format_for_editor')` to ensure the code remains functional on older WordPress installations. Provide the exact code block to replace and instructions on how to apply the fix, including ensuring no other deprecated filters or functions like `wp_htmledit_pre()` are still being used in that file

COMPUTER

What's happening

The [CKEditor for WordPress plugin's](#) `ckeditor_class.php` (in the `_editor()` method, around line 1128) registers the deprecated `wp_htmledit_pre()` and `wp_richedit_pre()` functions as callbacks on the `the_editor_content` filter. Both were deprecated in WordPress 4.3.0 in favor of `format_for_editor()`, which handles both the HTML and TinyMCE cases internally via its `$default_editor` argument ([WordPress Developer Resources](#), [WordPress core changeset](#)). A grep of the full file confirms these are the only two deprecated calls present — no other stray deprecated functions or filters exist in this file.

Exact code to replace

Find this block (original, ~line 1128):

```
if ('html' == wp_default_editor()) {
    add_filter('the_editor_content', 'wp_htmledit_pre');
} else {
    add_filter('the_editor_content', 'wp_richedit_pre');
    $switch_class = 'tmce-active';
}
```

Replace it with:

```
if ('html' != wp_default_editor()) {
    $switch_class = 'tmce-active';
}

if (function_exists('format_for_editor')) {
    // WP 4.3+: format_for_editor() replaces the deprecated
```

```
// wp_htmledit_pre() and wp_richedit_pre() functions.
add_filter('the_editor_content', 'format_for_editor', 10, 2);
} elseif ('html' == wp_default_editor()) {
// Back-compat for WordPress installs older than 4.3.
add_filter('the_editor_content', 'wp_htmledit_pre');
} else {
add_filter('the_editor_content', 'wp_richedit_pre');
}
```

Why it's written this way:

- `format_for_editor($content, $default_editor)` takes the same two arguments WordPress already passes through the `the_editor_content` filter, so `add_filter(..., 10, 2)` wires it up correctly for both HTML and Visual modes in one call.
- The `$switch_class` toggle (which controls whether the editor loads showing HTML or Visual mode) is pulled out of the filter-selection logic so it's set correctly regardless of which branch runs.
- The `function_exists('format_for_editor')` check means sites still on WordPress older than 4.3 silently fall back to the original `wp_htmledit_pre()`/`wp_richedit_pre()` behavior instead of fatal-erroring on an undefined function.

How to apply it

A ready-to-use unified diff (`ckeditor_class.patch`) was generated and shared. Two ways to apply it:

1. Via patch (fastest):

```
cd /path/to/wp-content/plugins/ckeditor-for-wordpress
patch -p1 --backup ckeditor_class.php < ckeditor_class.patch
```

--backup keeps a `.orig` copy in case you need to roll back.

2. Manual edit: open `ckeditor_class.php`, locate the `if ($set['quicktags'] && $set['tinymce'])` block, and swap in the replacement code shown above.

After patching, verify:

```
grep -n "wp_htmledit_pre\|wp_richedit_pre\|format_for_editor" ckeditor_class.php
```

You should see `format_for_editor` in the primary branch and the two old function names only inside the `elseif/else` fallback — confirming the deprecated calls are gated behind the compatibility check rather than running unconditionally.

Turn 2 — 19:49

USER

Check the rest of the plugin for other deprecated WordPress functions

COMPUTER

Full deprecated-function audit

The entire `ckeditor-for-wordpress` plugin (all 27+ PHP files, excluding the bundled third-party `ckeditor/` JS library) was cross-referenced against WordPress core's actual deprecated-function lists

(wp-includes/deprecated.php, wp-admin/includes/deprecated.php, wp-includes/pluggable-deprecated.php). Everything outside ckeditor_class.php — ckeditor_wordpress.php, includes/*.php, the bundled gd-star-rating and nggallery sub-plugins, and the file manager — came back clean. All findings are in ckeditor_class.php:

Line(s)	Function	Status	Notes
1129, 1131	wp_htmledit_pre(), wp_richedit_pre()	Already patched	Fixed with the format_for_editor() conditional applied in Turn 1
511–512	wp_tiny_mce	Dead legacy code	Only checked via has_action()/remove_action() — a hook name from pre-3.3 WordPress that no longer exists, so this branch is always inert on any modern WP install
517–520	wp_print_editor_js, wp_preload_dialogs, wp_tiny_mce	Dead legacy code	Same pattern — has_filter() checks against hooks that no longer fire in WordPress 3.3+
506–507	codepress_footer_js, wp_enqueue_script('codepress')	Already commented out	Dead code, never executes; references a function deprecated since WP 3.0.0 but has no runtime effect
1168	apply_filters('the_editor', ...)	Not actually an issue	Matched the scan because the_editor() is a deprecated *function* name in WP core, but the plugin calls apply_filters('the_editor', ...) — a filter hook, not the function. This exact hook is still fired today inside WordPress's own current _WP_Editors::editor() method, so it's safe and current.

Since has_action()/has_filter()/remove_action()/remove_filter() never actually call wp_tiny_mce(), wp_print_editor_js(), or wp_preload_dialogs(), none of these fire a _deprecated_function() notice — they're just checking whether ancient (pre-2011, WP < 3.3) hook registrations exist, which they never will on any currently-supported WordPress version. Functionally this code is harmless dead weight, not an active bug, but worth trimming for cleanliness.

Optional cleanup patch

remove_tinymce() and add_post_js() (lines 510–523) can be safely deleted or stubbed, since on WordPress 3.3+ they never do anything:

```
public function remove_tinymce() {
    // No-op: legacy pre-3.3 TinyMCE hooks (wp_tiny_mce) no longer exist.
}
```

```
public function add_post_js() {
// No-op: legacy pre-3.3 editor hooks (wp_tiny_mce, wp_print_editor_js,
// wp_preload_dialogs) no longer exist; the modern _WP_Editors class
// is used instead, so this compatibility shim is unnecessary.
}
```

One unrelated but worth-flagging item outside the scope of "WordPress" functions specifically:

plugins/nggallery/ckeditor.php (bundled NextGEN Gallery integration) defines class `add_ckeditor_button` with a same-named method `add_ckeditor_button()` as a PHP4-style constructor rather than `__construct()`. This isn't a WordPress deprecation — it's a PHP language-level one, deprecated since PHP 7 and fatal in PHP 8+.

Turn 3 — 20:00

USER

Apply all the patch changes to the files including the cruft removal and php 8 fix for `ckeditor.php` and generate a complete plugin archive in .zip format for download.

COMPUTER

All three fixes were applied to the full plugin source (cloned fresh from the [official repo](#)) and packaged as a zip archive. Summary of what changed:

``ckeditor_class.php``

- The `wp_htmledit_pre()`/`wp_richedit_pre()` filter registration is now gated behind `function_exists('format_for_editor')`, using `format_for_editor()` on WordPress 4.3+ and falling back to the old functions only on older installs.
- `remove_tinymce()` and `add_post_js()` are stubbed to no-ops (with explanatory comments) since their `wp_tiny_mce`, `wp_print_editor_js`, and `wp_preload_dialogs` hook checks never fire on WordPress 3.3+.
- The commented-out `codepress_footer_js()` reference in `file_editor_js()` was removed.

``plugins/nggallery/ckeditor.php``

- Renamed the PHP4-style constructor `add_ckeditor_button()` to `__construct()`, so the NextGEN Gallery button integration actually initializes on PHP 8+ instead of silently failing (PHP 8 removed automatic recognition of same-name-as-class constructors).

Brace balance was verified in both edited files, no active deprecated calls remained outside the intentional legacy-compat fallback and comments, and the zip was re-extracted to double-check the patched code was present before sharing. Everything else in the plugin (`ckeditor_wordpress.php`, `includes/`, `gd-star-rating`, the file manager) was already clean, so it was included unmodified.

Turn 4 — 20:55

USER

I uploaded this patched version of "ckeditor-for-wordpress" plugin and it functioned and eliminated the deprecated call complaint. There is a problem that remains the same. When "ckeditor-for-wordpress" plugin is enabled and I click on theme > customize it a fatal error is thrown. The following is the error message: Fatal error: Uncaught

Error: Call to undefined method `_WP_Editors::enqueue_default_editor()` in
/home/politico/amarkeplaceofideas.com/wp-includes/general-template.php:4013 Stack trace: #0
/home/politico/amarkeplaceofideas.com/wp-includes/widgets/class-wp-widget-text.php(435): wp_enqueue_editor()
#1 /home/politico/amarkeplaceofideas.com/wp-includes/class-wp-hook.php(341):
WP_Widget_Text->enqueue_admin_scripts("") #2
/home/politico/amarkeplaceofideas.com/wp-includes/class-wp-hook.php(365): WP_Hook->apply_filters(NULL,
Array) #3 /home/politico/amarkeplaceofideas.com/wp-includes/plugin.php(522): WP_Hook->do_action(Array) #4
/home/politico/amarkeplaceofideas.com/wp-includes/class-wp-customize-widgets.php(685):
do_action('admin_print_scr...') #5 /home/politico/amarkeplaceofideas.com/wp-includes/class-wp-hook.php(341):
WP_Customize_Widgets->print_scripts("") #6
/home/politico/amarkeplaceofideas.com/wp-includes/class-wp-hook.php(365): WP_Hook->apply_filters("", Array)
#7 /home/politico/amarkeplaceofideas.com/wp-includes/plugin.php(522): WP_Hook->do_action(Array) #8
/home/politico/amarkeplaceofideas.com/wp-admin/customize.php(180): do_action('customize_contr...') #9 {main}
thrown in /home/politico/amarkeplaceofideas.com/wp-includes/general-template.php on line 4013

COMPUTER

Root cause found

ckeditor_class.php includes its own private copy of WordPress's `_WP_Editors` class (a shim originally written to support WordPress versions older than 3.3, from before that class existed in core), and it's declared unconditionally — with no check for whether WordPress core already provides one.

Because the plugin's file is loaded very early (at plugin-load time), its stale `_WP_Editors` class gets defined first. WordPress core only loads its own copy lazily — `wp_editor()` and `wp_enqueue_editor()` in `wp-includes/general-template.php` both do `if (!class_exists('_WP_Editors', false)) { require .../class-wp-editor.php; }` before using it. Since the plugin's class already "exists" by then, core skips loading its real, up-to-date version and every later call falls through to the plugin's outdated one instead.

That old shim never got `enqueue_default_editor()` — a method WordPress core added later specifically so the Customizer's Text widget can lazy-load the editor. Hence: Call to undefined method `_WP_Editors::enqueue_default_editor()` the moment Theme > Customize is opened.

The fix (first attempt)

The plugin's `_WP_Editors` class declaration was wrapped in a guard that:

1. First tries to load WordPress core's real `wp-includes/class-wp-editor.php` if it hasn't been loaded yet.
2. Only falls back to the plugin's bundled legacy shim if core's file genuinely doesn't exist (i.e., a WordPress version older than 3.3 from 2011)

```
if (!class_exists('_WP_Editors')) {  
    if (defined('ABSPATH') && defined('WPINC') && file_exists(ABSPATH . WPINC . '/class-wp-editor.php'))  
    {  
        require_once ABSPATH . WPINC . '/class-wp-editor.php';  
    }  
}  
  
if (!class_exists('_WP_Editors')) {  
    final class _WP_Editors {  
        // ... existing shim body unchanged ...  
    }  
} // end if (!class_exists('_WP_Editors'))
```

This means on the site's actual WordPress version, core's genuine, fully-current `_WP_Editors` class (with `enqueue_default_editor()` and everything else) loads normally, and the plugin's old shim never activates at all.

Validation

A PHP CLI was installed in the sandbox and `php -l` was run against every PHP file in the plugin (not just brace-counting) — all reported "No syntax errors detected," including the modified `ckeditor_class.php`. The rebuilt zip was re-extracted and re-linted to confirm the fix made it into the archive correctly before sharing.

(Note: this fix was reverted in Turn 5 after user feedback that it broke CKEditor loading — see below.)

Turn 5 — 21:39

USER

That patch resulted in the ckeditor not being loaded in. I was forced to use the "classic editor" wordpress plugin to get "ckeditor-for-wordpress" to function previously to all of these changes.

COMPUTER

What went wrong with the previous fix

The Turn 4 diagnosis was incomplete. `ckeditor_class.php` doesn't just borrow WordPress's `_WP_Editors` class — it entirely replaces it. The plugin defines its own `final class _WP_Editors` with a custom `editor()` method that outputs CKEditor's markup instead of TinyMCE's. That class-replacement trick is literally how the plugin takes over the classic editor screen.

The previous patch made the plugin defer to WordPress core's real `_WP_Editors` class whenever it's available — which is now every modern install. That fixed the Customizer crash, but it also meant core's real (TinyMCE-based) `editor()` method ran instead of the plugin's CKEditor version everywhere, including the main post editor. That's why CKEditor stopped loading and Classic Editor was needed just to get a usable rich-text field at all.

The corrected fix

Instead of stepping aside for core's class, the plugin's own `_WP_Editors` override was kept fully intact — so CKEditor keeps loading everywhere it always has — and just the one specific missing method was added: `enqueue_default_editor()`. This is the method WordPress added in version 4.8 specifically so the Customizer's Text widget can load the editor lazily (after the widget panel expands), and it's the one the crash was pointing at.

The new method enqueues CKEditor's own scripts (reusing the plugin's existing `editor_js()` logic) instead of falling back to TinyMCE, and includes the same `did_action('wp_enqueue_editor')` guard core uses to avoid double-loading.

WordPress core's other calls into `_WP_Editors` (the AJAX link-search handler, the widgets class, `media.php`) were checked to confirm no other externally-called method is missing from the plugin's shim — the plugin's `wp_link_query()` was already present and covers the AJAX case, and the remaining core methods (`wp_link_dialog()`, `default_settings()`, etc.) are only ever called from inside core's own `editor()`, which the plugin's version doesn't invoke — so they aren't reachable and won't fatal.

Validated with a real PHP 8.5 linter (installed in the sandbox) against every file in the archive, not just brace-counting, and confirmed the fix was present in the rebuilt zip before sharing.

Turn 6 — 22:24

USER

The "classic editor" plugin must be installed and activated for "ckeditor-for-wordpress" plugin to load. This is for your information only. Please take no action on this until instructed. The version of CKEditor used in this plugin is quite old. Please evaluate for me the practicality of updating the plugin to work with current version of ckeditor 5 LTS.

COMPUTER

Short answer

Not practical as an incremental patch. Upgrading this plugin to CKEditor 5 would essentially mean throwing out its entire front-end integration and rebuilding it from scratch — not a version bump. The PHP-side trick (overriding `_WP_Editors`) is reusable, but everything downstream of that is a different technology.

Why it's not a simple upgrade

Completely different JS architecture. CKEditor 4 ships as a single drop-in `ckeditor.js` file the plugin can just `wp_enqueue_script()`. CKEditor 5 is a modular, ES-module-based framework — you either build a custom bundle with webpack/Vite or use a "predefined build" (CDN/ZIP), and even those predefined builds were officially deprecated by CKSource in favor of package-based builds you assemble yourself ([CKEditor 5 predefined builds migration guide](#), [GitHub discussion on installation complexity](#)). There's no equivalent to CKEditor 4's "grab one file, done" model anymore.

Plugin API is not compatible. CKEditor 4's plugin system (`CKEDITOR.plugins.add`, `dialogs`, `widgets`) has nothing in common with CKEditor 5's Model/View/Command architecture. The `plugins/nggallery/ckeditor.php` integration, and any other custom toolbar buttons this plugin adds, would need to be rewritten from zero for CKEditor 5, not ported ([official CKEditor 4→5 migration guide](#) confirms there's no automated migration path for custom plugins).

Source/HTML view works differently. The classic HTML-toggle switcher this plugin relies on has no direct CKEditor 5 equivalent — you'd need the `SourceEditing` plugin, which behaves as an inline toggle rather than a separate textarea, and image/media upload would need a brand-new `UploadAdapter` wired into `wp.media`.

Licensing is a minor, solvable wrinkle, not a blocker. CKEditor 5 core is GPL-2.0-or-later, so it's license-compatible with a GPL WordPress plugin, but self-hosted use requires explicitly setting `licenseKey: 'GPL'` in the editor config since v44 (Oct 2024) — omitting it is now a hard error ([CKEditor 5 licensing docs](#), [license key docs](#)). The LTS Edition itself is only free during its "Active phase" (currently v47.x); once a version moves into "Maintenance phase," further patches require a paid subscription ([CKEditor 5 LTS docs](#)) — meaning free-tier LTS coverage is time-limited, not indefinite like CKEditor 4 was.

What you'd be signing up for

- Full rewrite of the plugin's JS bundle/build pipeline and asset enqueueing
- Full rewrite of any custom toolbar plugins (nggallery button, etc.) against a new API
- New upload adapter for WP media library integration
- New source/HTML-view handling
- Re-solving the exact editor-takeover problem just fixed for CKEditor 4 (Classic Editor plugin dependency, Customizer's `_WP_Editors::enqueue_default_editor()` call) — CKEditor 5 doesn't hook into `_WP_Editors` at

all, so this becomes a fresh integration design, not a port of the existing PHP layer

- Realistically this is "build a new WordPress editor plugin using CKEditor 5," reusing only the general idea (not the code) of the current plugin

Is there an existing shortcut?

No actively maintained "CKEditor 5 for WordPress" plugin that does what this one does (replace the main post editor) exists. The WordPress.org listings found only use CKEditor 5 for isolated custom admin fields inside other plugins (e.g. a property-description field), not as a general post-editor replacement ([example](#)). There's no "buy" option here; it would be fully custom "build."

Security context

CKEditor 4 hit end-of-life in June 2023, and the last free release (4.22.1) has no further security patches ([CKEditor 4 EOL announcement](#)). That's a real, valid concern — but a full CKEditor 5 rewrite is a disproportionate way to address it for a low-traffic/internal admin editing surface.

Recommendation

Given the effort (essentially a from-scratch plugin build) versus the value (a working, patched CKEditor 4 setup already running), lower-effort alternatives were suggested before committing to a CKEditor 5 port:

1. Do nothing for now — accept the EOL risk if this is an internal/trusted-user admin surface.
2. Drop CKEditor entirely and use WordPress's native Gutenberg editor.
3. Standardize on Classic Editor's built-in TinyMCE (already confirmed working) instead of CKEditor — no custom code needed.
4. Commission a genuinely new CKEditor 5 WordPress integration — only worth it for a hard feature requirement nothing else satisfies.

Turn 7 — 22:35

USER

What about updating the "ckeditor-for-wordpress" plugin to work with version 4.22.1? What do you think of that idea?

COMPUTER

Short answer

Yes — this one is genuinely practical, and much lower-risk than a CKEditor 5 migration. It's a same-major-version update (4.5.3 → 4.22.1), not an architecture change.

What was found inspecting the current plugin

Read-only inspection (no edits) confirmed:

- The bundled `ckeditor/` folder in the plugin is a pristine, unmodified vendor copy of CKEditor **4.5.3** (from 2015) — `ckeditor/config.js` is just stock CKSource boilerplate, confirming no custom patches were made inside the vendored editor itself.
- All of the plugin's WordPress-specific customizations live **outside** that folder, as standard external CKEditor plugins (`plugins/nggallery`, `wpgallery`, `mediaembed`, `wppolls`, `gd-star-rating`, `linkbrowser`, `wpmore`, `vvq`),

loaded via the standard `extraPlugins / CKEDITOR.plugins.addExternal()` mechanism.

- The toolbar/button config (`ckeditor.config.js`) only references long-standing, still-current CKEditor 4 buttons (Save, NewPage, Preview, Templates, PageBreak, Styles, etc.).
- Skin selection is auto-discovered at runtime from whatever folders exist under `skins/` — it's not hardcoded to a specific skin name, so a newer skin set drops in without code changes.

That combination — pristine vendored core + external-plugin API for everything custom — is exactly the situation where a version bump is low-risk. CKEditor 4's plugin/dialog/widget API has been stable across the entire v4.x line since roughly 2012.

What the update would involve

1. Download the official final release, CKEditor **4.22.1 "Full"** package, from CKSource.
2. Replace the plugin's `ckeditor/` directory wholesale with the new package contents.
3. Leave `ckeditor.config.js` and the 8 custom plugin folders as-is (they don't touch the vendored core).
4. Smoke-test each custom plugin folder in the WP admin.

Licensing — no complication

CKEditor 4 kept its GPL/LGPL/MPL tri-license all the way through the final 4.22.1 release ([CKEditor 4 licensing docs](#)). No license-key requirement like CKEditor 5.

Security reality check

4.22.1 gives roughly 8 years and hundreds of bug/security fixes over the currently-bundled 4.5.3, and it's the officially final, most-hardened free release ([CKEditor 4.22.0/4.22.1 release notes](#)). But CKSource's End-of-Life announcement means no further official patches ship at all after this version — and known vulnerabilities discovered *after* EOL affect 4.22.1 itself (e.g. CVE-2024-24815, CVE-2024-24816, CVE-2024-43411), patched only unofficially by Linux distro maintainers, not by CKSource upstream ([Ubuntu ckeditor security changelog](#)).

Recommendation

A clearly worthwhile, low-risk improvement — a much better use of effort than the CKEditor 5 rewrite path.

Turn 8 — 22:44

USER

Build the updated package with 4.22.1 as you described.

COMPUTER

The updated package contains everything from the previous patched build (the `format_for_editor()` fix, dead-code cleanup, the PHP 8 constructor fix, and the `_WP_Editors::enqueue_default_editor()` Customizer fix), plus the CKEditor core bump from 4.5.3 → 4.22.1.

What changed in this update

- **Replaced the entire vendored ``ckeditor/`` directory** with the official, final CKEditor 4.22.1 "Full" package ([download source](#)) — brings ~8 years of bug and security fixes.
- **Trimmed dev/demo cruft** not used by the WordPress integration (`samples/`, `adapters/`, `bender-runner.config.json`, `build-config.js`) — matches how the plugin's original 4.5.3 bundle was

packaged.

- **Caught and fixed one real compatibility gap:** the official 4.22.1 package only ships the `moono-lisa` skin — the `moono` skin (hardcoded as this plugin's default, and also offered as `kama`) was fully retired starting with CKEditor 4.6.0. Both legacy skins (`moono`, `kama`) were carried forward unmodified alongside the new `moono-lisa`, so any site with an existing saved skin preference keeps rendering exactly as before, with zero PHP changes required.
- **Verified no breakage:** checked every custom sub-plugin (`nggallery`, `wpgallery`, `mediaembed`, `wppolls`, `gd-star-rating`, `linkbrowser`, `wpmore`, `vvq`, `wpeditimage`) against the new plugin set, confirmed all toolbar-referenced buttons (`Image`, `Link`, `SpecialChar`, `Table`, `PageBreak`, `Templates`, `Div`, `Forms`, `Scayt`) still resolve, and ran `php -l` across the whole plugin — all clean. No PHP files were touched in this step.

Licensing is unaffected — CKEditor 4.22.1 is still GPL/LGPL/MPL tri-licensed, same as before.

Worth remembering: 4.22.1 is CKSource's officially final free release — there are already a few post-EOL CVEs against it that only get unofficial distro-level patches, not upstream ones. It's a real improvement over 4.5.3, but not a return to active vendor support.

Turn 9 — 23:19

USER

I see that there is a `ckeditor` version 4.25.2-lts available.

COMPUTER

What 4.25.2-lts actually is

CKSource introduced **CKEditor 4 LTS ("Extended Support Model")** specifically because 4.22.1 was the last free release. Every version from **4.23.0-lts onward, including 4.25.2-lts, requires a purchased "Extended Support Model" subscription and a license key** — there's no free download or open-source path to it ([CKEditor 4 licensing page](#), [CKEditor 4 npm package notes](#), [ckeditor4 GitHub repo notice](#)). The changelog states plainly that releases from 4.23.0-lts up are "only available to customers who decided to acquire the LTS version of the editor" ([CKEditor 4 changelog](#)).

Concretely, using it would mean:

- Contacting CKSource sales and paying for the Extended Support Model.
- Adding a `licenseKey` config value to activate the editor.
- Support running through December 2028 under that subscription ([CKEditor 4 Extended Support Model docs](#)).

It cannot be obtained or bundled for free — it's not something that can just be downloaded.

Is it worth pursuing?

- **4.24.0-lts:** fixed a real core Advanced Content Filter bypass (CDATA-detection XSS) — matters regardless of environment — plus two XSS issues specifically in the bundled `samples/` demo pages ([Drupal.org security advisory notes](#)).
- **4.25.0-lts:** fixed a low-risk theoretical domain-takeover XSS, and removed the GeSHi syntax highlighter from the Code Snippet plugin over an XSS concern ([CKEditor blog post](#)).
- **4.25.2-lts:** purely administrative (license-header/readme updates) — no security content at all.

The `samples/` folder had already been stripped from the built package, removing the attack surface for two of the three sample-only XSS issues, and the Code Snippet/GeSHi plugin isn't used by this plugin either. The one genuinely relevant gap left open is the core ACF/CDATA bypass fixed in 4.24.0-lts — a real vulnerability, but paywalled behind the paid LTS tier.

Bottom line

Given the cost of an ESM subscription versus how narrow the remaining exposure is, this wasn't recommended purely for this legacy WordPress plugin — that budget would likely be better spent eventually retiring CKEditor 4 entirely.

Turn 10 — 23:23

USER

The ckeditor version 4.22.1 puts up a nag message that says "This CKEditor 4.22.1 version is not secure. Consider [upgrading to the latest one](#), 4.25.2-lts". Is there a way to remove that?

COMPUTER

Yes — this is a legitimate, documented CKEditor config option, not something you need to hack around.

What's causing it

Version 4.22.0/4.22.1 added a built-in "version check" notification system specifically so integrators would be nagged to upgrade after CKEditor 4 hit end-of-life. It's controlled by the `versionCheck` config setting, which defaults to `true` ([official CKEditor blog post](#), [CKEditor 4 config API docs](#)).

The fix

Setting `config.versionCheck = false;` in the editor's config removes the message entirely, and also stops the editor from pinging CKSource's servers on every page load to perform that check ([CKEditor 4 source](#)).

For this plugin specifically, the right place to add it is the plugin's own `ckeditor.config.js` — the WP-integration config file the plugin author already designates for customizations (survives any future CKEditor core version swap).

Being straight about the tradeoff

Disabling `versionCheck` only silences the message — it doesn't patch anything. The site would still be on 4.22.1 with the one known unpatched core issue (the ACF/CDATA bypass fixed only in the paid 4.24.0-lts). This change doesn't affect that either way.

Turn 11 — 23:26

USER

Please rebuild the zip with "versioncheck=false"

COMPUTER

`config.versionCheck = false;` was added to `ckeditor.config.js` — the WP-integration config file the plugin already loads as `customConfig`, so this survives future CKEditor core swaps. Everything else is unchanged from the last package (still CKEditor 4.22.1 core, legacy skins preserved, all prior PHP fixes intact). Verified the setting is present in the zip, the core version is still 4.22.1, and the file parses cleanly with no JS syntax errors.

The code added, with a comment explaining the tradeoff:

```
// Disable the "this version is not secure" nag notification introduced in 4.22.
// This only silences the notice/HTTP check; it does not patch any underlying
// vulnerabilities. See
https://ckeditor.com/docs/ckeditor4/latest/api/CKEDITOR_config.html#cfg-versionCheck
config.versionCheck = false;
```

Worth repeating: this only hides the notice and stops the version-check network call — it doesn't patch anything. The one real gap flagged earlier (the ACF/CDATA bypass fixed only in the paid 4.24.0-lts) is unaffected either way.

Turn 12 — 23:40

USER

I would like to edit the description and version of the "ckeditor-for-wordpress" that comes up on my wordpress installation plugin page. What file and line number is that on?

COMPUTER

That's the one WordPress reads for the Plugins page.

File and lines

``ckeditor_wordpress.php`` (the plugin's main file, at the root of the plugin folder), lines 3–10 — the standard WordPress plugin header comment block:

- **Line 6** — Description: Replaces the default WordPress editor with `<a href="http://ckeditor.com/"> CKEditor</a>`
- **Line 7** — Version: 4.5.3.3

This block is what WordPress parses to populate the name, version, and description shown on the **Plugins** admin page — it has nothing to do with the CKEditor core version bundled (currently 4.22.1); that's just the plugin's own self-reported version number, still left at its original 4.5.3.3 from before any of the patches.

One thing to note

There's also a `readme.txt` at the plugin root with its own `Stable` tag: and description fields near the top — WordPress only reads that file for plugins installed *from* the official WordPress.org repository (to show the `readme` tab/changelog). Since a manually-uploaded zip is being used, only the header in `ckeditor_wordpress.php` actually drives what appears on the Plugins page.

End of session transcript.